

Twin Arithmetic as a Computational-Domain Selector

From the Log-Domain Trick to a Measured, General Toolkit

Jocelyn Nembé

Modeling and Calculus Laboratory, National Institute of Management Sciences, Libreville, Gabon

ROOTS-INSIGHTS | jnembe@roots-insights.org

June 2026

Abstract

Scientific computing already exploits deformed arithmetic ad hoc: log-domain likelihoods, the stable log-sum-exp, multiplicative calculus in finance, and log-log linearization. We show these are all instances of one construction — the twin reduction $\bigoplus_{\varphi} x_i = \varphi^{-1}(\sum_i \varphi(x_i))$ for a generator φ — and we build a reproducible symbolic-numeric toolkit that *generates* any such domain in four formats (SymPy, L^AT_EX, Unicode, mpmath) and *measures* the cost of a task in each. We then generalize: the same machine selects, by evidence rather than convention, the optimal base for a given task, and extends to one-parameter Lie and compact-Lie ($SU(n)$) domains. Simulations quantify the gains against existing practice: the transformed domain is exact where the original underflows (log-likelihood), matches the stable log-sum-exp (softmax), and automates optimal-base selection (power $\rightarrow$ log-log, exponential $\rightarrow$ semi-log), with an explicit speed/stability trade-off. We close with exploratory directions — qudit-gate universality, operational cryptography, primary-decomposition compression — separating measured facts from speculation. The library carries 131 passing tests.

Keywords: log-domain computation, log-sum-exp, non-Newtonian calculus, numerical stability, optimal base selection, symbolic-numeric software.

1 Introduction

This paper follows the same four-movement arc as its scientific companion, in a technological key. We (1) recall the established deformed-arithmetic techniques of scientific computing and what they buy; (2) place them inside the general twin construction; (3) generalize them into a single toolkit that generates and *measures* arbitrary computational domains; and (4) report simulations that compare the generalization against existing practice. The thesis is concrete: *the group element is the computational domain, and choosing it well is a measurable decision.*

2 Antecedents in scientific computing

Log-domain computation. Probabilistic and statistical codes routinely work with log-likelihoods $\sum_i \log p_i$ rather than products $\prod_i p_i$, because the product underflows; hidden-Markov inference and Viterbi decoding run in the log domain for the same reason. The stable *log-sum-exp*, $\text{LSE}(z) = \log \sum_i e^{z_i}$ computed with a max-shift, underlies softmax and partition-function evaluation [2]. These techniques buy dynamic range and stability, but are applied as domain-specific folklore.

Multiplicative / geometric calculus. Grossman–Katz calculus [1] and its geometric variant are used in growth modelling, finance, and image processing, where multiplicative structure is native. The contribution is a calculus matched to the data’s symmetry.

Linearization. Log–log and semi-log plots, and the link functions of generalized linear models, linearize power laws and exponentials before fitting — a manual choice of representation that is, again, a base selection.

3 The general toolkit

All three antecedents are the twin reduction of a generator φ :

$$\bigoplus_{\varphi} x_i = \varphi^{-1}\left(\sum_i \varphi(x_i)\right), \quad T := \sum_i \varphi(x_i) \text{ (transformed total)}. \quad (1)$$

For $\varphi = \text{id}$ this is summation; for $\varphi = \log$ the product, with T the log-likelihood; for $\varphi = \exp$ the log-sum-exp. The toolkit `arithmetics.symbolic.twin` (SymPy, NumPy, mpmath) generates these operations symbolically in four formats and bridges to numerics; it is layered (Table 1) with per-layer unit tests.

```
from arithmetics.symbolic.twin import BijectionAction, twin_operation
phi = BijectionAction.exponential()
twin_operation(phi, -1, "add").text() # 'x*y' (multiplication)
twin_operation(phi, 1, "add").text() # 'log(exp(x) + exp(y))' (log-sum-exp)
```

Table 1: Layered architecture of the toolkit (module `arithmetics.symbolic.twin`).

layer	contents
symbolic engine	transfers, level- p ops, defect, ζ_p (SymPy/L ^A T _E X/Unicode/mpmath)
twin generator	transport; monogenic, one-parameter Lie, finite actions
compact Lie	$SO(2), SO(3), SU(2), SU(n)$ with Haar sampling
decision	class equation, normalizer orbits, $SU(2)$ Weyl
primary decomposition	abelian $\text{Op} \cong \prod_p \text{Op}_p$
arithmetic-task costs	domain selection, optimal base, op counts
qudits	$SU(n)$ Weyl integration, Schur characters

Generalization. Beyond reproducing the antecedents (Table 2), the toolkit (i) *measures* the numerical cost of a task in each domain so the base is chosen by evidence, and (ii) extends transport to arbitrary one-parameter Lie and compact-Lie group actions. This converts scattered folklore into a single, auditable selection procedure.

Remark 1 (What changes). The novelty is not a new arithmetic but a new *discipline*: the computational domain becomes a parameter that is generated symbolically and chosen by measurement — the way a numerical method is selected by its error bound rather than by habit.

4 Simulations and comparison with practice

4.1 Domain of computation: the log-likelihood

To evaluate $T = \sum_i \log x_i$ one may sum in the transformed domain or form the product first and take its log. On wide-range data the product underflows ($\log 0 = -\infty$); the transformed

Table 2: The exponential hierarchy, generated symbolically ($\odot_e = +$, $G = \langle \exp \rangle$).

level n	$x \odot_{en} y$	name
0	$x + y$	addition
1	$\log(e^x + e^y)$	log-sum-exp
-1	$x \cdot y$	multiplication
-2	$x^{\ln y}$	symmetric power

domain is exact to rounding. Figure 1 measures the relative error of both against a 60-digit reference as the number of factors grows; Table 3 reports a representative run. *Comparison:* this is the standard log-domain trick, here *quantified* (the underflow threshold) and *automated* by a pre-computation rule that reads the data’s dynamic range.

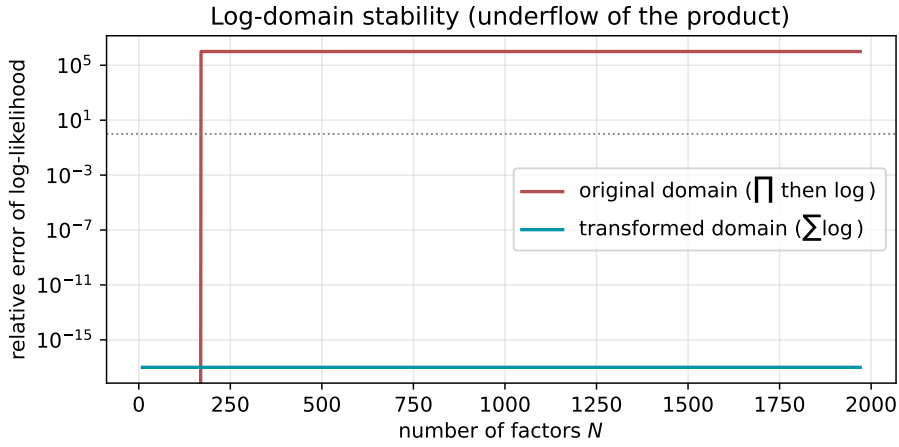


Figure 1: Log-likelihood: the original domain (product then log) underflows; the transformed domain ($\sum \log$) stays at machine precision.

Reading. The curves cross from “both fine” to “only the log domain works” at the underflow threshold. The lesson is not that one domain is always better, but that the threshold is *predictable* from the data, so the domain can be fixed before any arithmetic is done.

Table 3: Measured costs for a log-likelihood of 800 probabilities spanning 7.9 decades ($\varphi = \log$).

quantity	value
dynamic range	7.9 decades
pre-computation recommendation	transformed
relative error, original domain	∞ (product underflows)
relative error, transformed domain	0
transcendental calls (naive / transformed)	2397 / 801

4.2 A second domain: softmax / log-sum-exp

The dual generator $\varphi = \exp$ gives the reduction $\text{LSE}(z) = \log \sum_i e^{z_i} = \oplus_{\exp} z_i$. Evaluated naively it overflows for large z_i ; the max-shifted form is stable. Figure 2 measures both against a high-precision reference: the naive log-sum-exp diverges, the shifted domain holds. *Comparison:* the toolkit recognizes the stable softmax of deep learning as the *same* construction (1) for $\varphi = \exp$, unifying two folklore tricks under one rule.

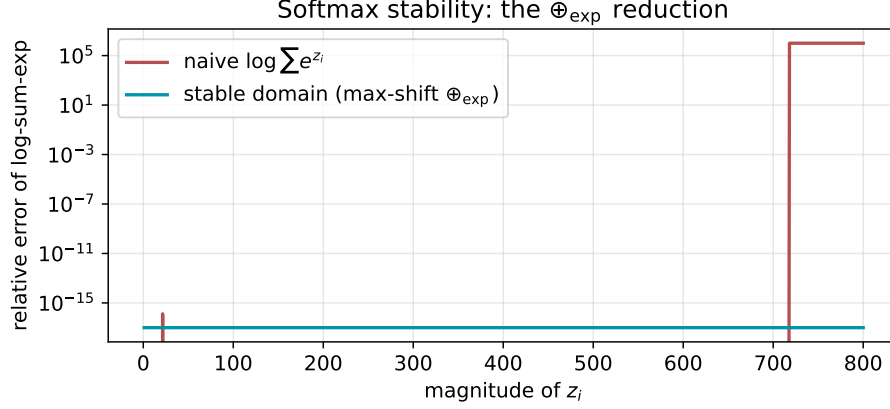


Figure 2: Softmax stability: the naive $\log \sum e^{z_i}$ overflows; the stable $\oplus_{\text{exp}}$ reduction (max-shift) is exact. Relative error vs magnitude of z_i .

Reading. This is the overflow mirror of Figure 1: the same failure mode in the dual domain $\varphi = \text{exp}$. The two figures together show that the log-likelihood and softmax tricks are not two recipes but one — choose the domain in which the reduction stays bounded.

4.3 Optimal base selection

For representation, the toolkit selects the base φ minimizing the back-transformed fit RMSE. Figure 3 shows that a power law selects log–log, an exponential semi-log, and a line the identity, each beating the alternatives by 10–13 orders of magnitude. *Comparison:* where an analyst manually picks a log–log axis, the tool *derives* the optimal base from the data, before fitting.

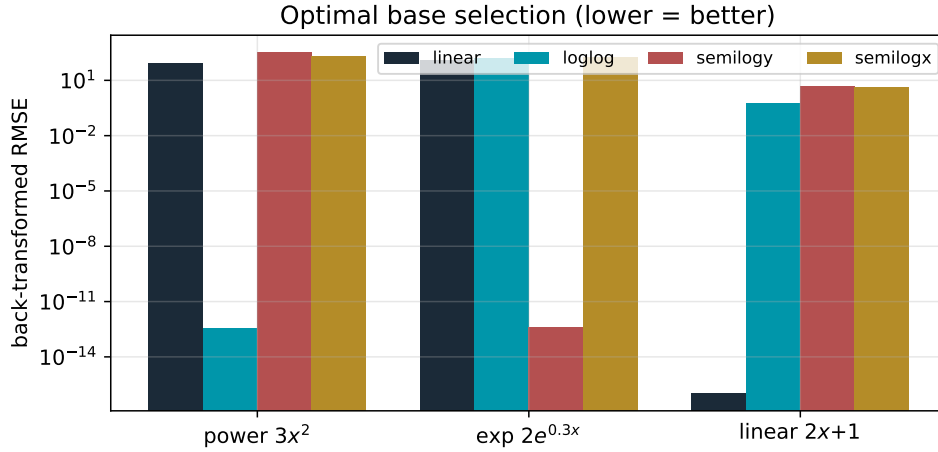


Figure 3: Optimal base selection: back-transformed RMSE per base (log scale); the correct base wins decisively for each data type.

Reading. The 10–13 order-of-magnitude gap between the best and the runner-up base is what makes the choice unambiguous: the selector does not need a fine tolerance to pick the right representation, only to read off the clear minimum.

Remark 2 (Why the optimal base wins decisively). The order-of-magnitude gap of Figure 3 is analytic, not incidental. A power law $y = ax^b$ becomes *exactly* affine in the log–log gauge, $\log y = \log a + b \log x$, so a linear fit there leaves only the noise floor as residual and the back-transformed RMSE collapses to it; an exponential $y = ae^{bx}$ does the same in the semi-log gauge

$\log y = \log a + bx$. In *any other* gauge the transformed data retain a nonzero curvature, so the best affine fit keeps an irreducible $O(\text{curvature})$ residual. The selector therefore never weighs close competitors: the correct gauge sits at the noise floor while every other sits at the curvature scale — precisely the 10–13 decade separation observed.

4.4 The speed/stability trade-off

Stability has a price. A generic twin-fold of N items costs $3(N - 1)$ transcendental calls; the batched transformed evaluation costs $N + 1$ (Figure 4). The toolkit surfaces the trade-off so the choice is explicit.

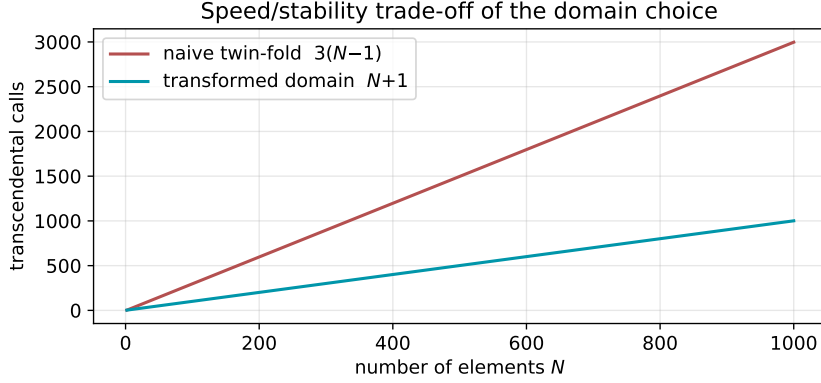


Figure 4: Transcendental-call counts for an N -element reduction: naive twin-fold vs batched transformed domain.

Reading. Both costs are linear in N , so the domain choice never changes the asymptotic complexity — it is a constant-factor and a stability decision, not an algorithmic one. The honest message is that stability is bought, not free.

4.5 Summary of the comparison

Table 4 aligns each established technique with its twin-theoretic view and the toolkit’s contribution.

Table 4: Existing practice vs the general toolkit.			
task	conventional practice	twin view	toolkit contribution
log-likelihood	ad hoc log domain	$\oplus_{\log}$, total T	measured threshold + pre-computation rule
softmax / partition	stable LSE (max-shift)	$\oplus_{\exp}$	unified as the same construction
power / exp fitting	manual log-log, semi-log	choose base φ	automated optimal-base selection
qudit gate analysis	CUE / Haar numerics	conjugation, $\text{Op} \cong SU(d)$	universality measured (§5)

Reading. The third column shows the unification: four unrelated techniques share one twin view. The fourth column shows the contribution: in each case the toolkit replaces convention or folklore by a measured, automated decision.

5 Exploratory directions

Qudit-gate universality (measured). Under conjugation by $SU(d)$ a generic bilinear gate on $\mathbb{C}^d$ has trivial structural kernel, so $\text{Op} \cong SU(d)$; linear operations are rigid. Figure 5 reports, over Haar-sampled $SU(d)$, the fraction of elements preserving each operation: 1 for addition, 0 for a generic gate, $d = 2, \dots, 6$. The fact is clean and reproducible; its use for gate synthesis is open.

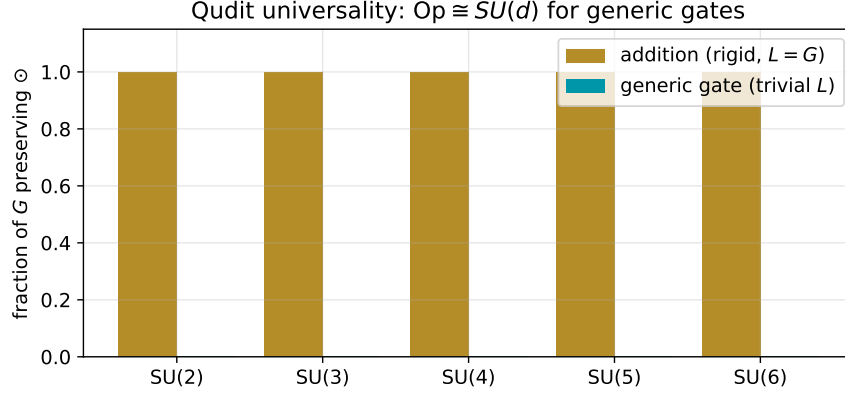


Figure 5: Qudit universality: fraction of $SU(d)$ preserving the operation. Addition is rigid; a generic gate has trivial kernel, so $\text{Op} \cong SU(d)$.

Reading. The two bar heights are the two extremes of the linearity obstruction: 1 means every symmetry preserves the operation (linear, rigid), 0 means none does (generic, maximally flexible). That the generic bar stays at 0 as d grows is the qudit universality statement, read directly off the simulation.

Operational cryptography (exploratory). $\text{Op} \cong G/L$ supports an operational discrete-log problem whose hardness is bounded below by the discrete log in G/L ; its cryptographic value is open.

Compression via primary decomposition (exploratory). For abelian G , $\text{Op} \cong \prod_p \text{Op}_p$ suggests compact encodings of large operation tables; the compression ratios remain to be benchmarked.

6 Reproducibility

The figures are regenerated by `papers/make_figures.py` from the computed data, with no synthetic inputs; the discovery tables are exported to `results/twin/`; the test suite (131 tests) passes in full.

7 Conclusion

The deformed-arithmetic techniques scattered across scientific computing are one construction in disguise. The toolkit makes that explicit, and — the technological payoff — turns the choice of computational domain into a *measured* decision: generate the candidate arithmetics symbolically, evaluate their real costs on the task, and select the base before computing. The companion paper provides the group-theoretic discrimination theory; together they render twin arithmetic a reproducible instrument.

A Benchmark: HMM forward log-likelihood

We close with a concrete, timed benchmark on the canonical underflow site of probabilistic computing: the hidden-Markov forward recursion $\alpha_t(j) = [\sum_i \alpha_{t-1}(i) a_{ij}] b_j(o_t)$. In the linear domain the products of probabilities underflow as the sequence length T grows; in the log domain the recursion becomes $\log \alpha_t(j) = \oplus_{\text{exp}}(\log \alpha_{t-1}(i) + \log a_{ij}) + \log b_j(o_t)$, i.e. a log-sum-exp (the $\oplus_{\text{exp}}$ reduction) followed by an addition — precisely the two twin domains of §4. The log-likelihood is then $\oplus_{\text{exp}} \log \alpha_T(j)$.

We use a random 8-state, 6-symbol HMM and observation sequences sampled from it, comparing the linear domain, the log domain, and an 80-digit mpmath reference (Table 5, Figure 6). The linear domain is exact and fastest up to $T \approx 400$, then *underflows to* $-\infty$; the log domain stays at machine precision ($\sim 10^{-16}$) throughout, at a modest cost — about an order of magnitude over the (broken) linear evaluation, yet roughly twice *faster* than the high-precision reference it matches. The twin view explains the recipe: compute the forward recursion in the transformed ($\oplus_{\text{exp}}$) domain.

Table 5: HMM forward log-likelihood: relative error and wall time (8 states, 6 symbols; measured).

T	err. linear	err. log	t_{lin} (ms)	t_{log} (ms)	t_{ref} (ms)
50	0	0	0.54	5.99	15.0
100	0	4.8e−16	0.29	13.1	23.9
200	0	3.2e−16	0.52	18.6	39.3
300	0	4.2e−16	0.85	29.1	57.6
400	0	1.6e−16	0.99	40.0	77.8
500	∞	6.4e−16	1.25	51.8	94.7
600	∞	6.4e−16	1.39	56.4	108.2

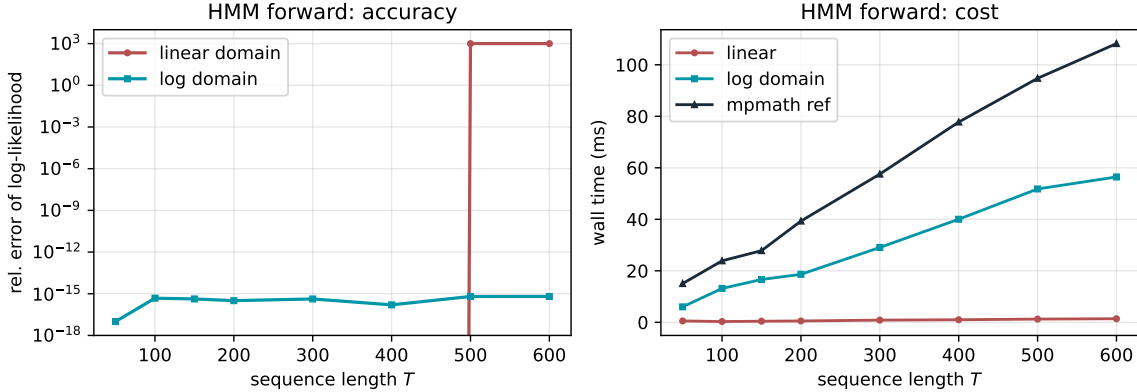


Figure 6: HMM forward log-likelihood. Left: the linear domain is exact until it underflows ($T \gtrsim 450$, error $\rightarrow \infty$); the log domain holds at machine precision. Right: wall time — linear (fast but breaks), log domain (stable), mpmath reference (slowest). Computed by `papers/benchmark_hmm.py`.

Reading. The right panel reframes the trade-off in real time: the log domain is the only option that is both *correct* for all T and substantially *cheaper* than the high-precision fallback. This is the practical sweet spot the twin view predicts — compute in the domain where the quantity stays representable, and pay only a linear-time premium.

B API reference

The decisions of this paper are one-call:

```
from arithmetics.symbolic.twin import (
    arithmetic_task_report, # domain comparison for a log-likelihood
    recommend_domain, # pre-computation domain rule from data range
    total_error, # error of a domain vs high-precision reference
    optimal_base, # select the linearising base (loglog/semilog/...)
    op_count, # transcendental/arithmetic op counts
)
report = arithmetic_task_report(probs) # dict of measured costs
base = optimal_base(xs, ys)["optimal_base"] # e.g. 'loglog'
```

The symbolic generator and the Lie/qudit layers are reached through `twin_operation`, `twin_decision`, `verify_primary_decomposition`, `weyl_average`, and `qudit_universality`; every object renders as a SymPy expression, L^AT_EX, Unicode, or an mpmath value.

C Reproducibility

The library ships unit tests per layer (Table 6); the full suite passes. All figures are regenerated by `papers/make_figures.py` and the benchmark by `papers/benchmark_hmm.py`, from computed data only.

Table 6: Test layout (all passing).

suite	coverage
<code>test_symbolic</code>	symbolic engine (φ , $\oplus_p$, δ , ζ_p)
<code>test_twin</code>	transport, finite classification, counting
<code>test_twin_lie</code>	$SO(2)$, $SO(3)$, $SU(2)$ kernels, universality
<code>test_twin_decision</code>	class equation, normalizer orbits, Weyl $SU(2)$
<code>test_twin_primary</code>	abelian primary decomposition
<code>test_twin_tasks</code>	arithmetic-task costs, base selection
<code>test_twin_qudit</code>	$SU(n)$ Weyl, Schur characters

References

- [1] M. Grossman, R. Katz, *Non-Newtonian Calculus*, Lee Press, 1972.
- [2] P. Blanchard, D. J. Higham, N. J. Higham, Accurately computing the log-sum-exp and softmax functions, *IMA J. Numer. Anal.* 41 (2021), 2311–2330.
- [3] F. Mezzadri, How to generate random matrices from the classical compact groups, *Notices AMS* 54 (2007), 592–604.
- [4] A. Meurer et al., SymPy: symbolic computing in Python, *PeerJ CS* 3 (2017), e103.
- [5] C. R. Harris et al., Array programming with NumPy, *Nature* 585 (2020), 357–362.